

大语言模型+智能体（代码生成与运行）

实验背景

大语言模型（Large Language Models, LLMs）在自然语言处理领域取得了突破性进展，但其能力并不局限于文本生成。通过与外部工具（External Tools）的结合，LLM 能够突破纯文本交互的局限，实现对结构化数据的处理、复杂计算的执行以及动态环境的感知与操作。这种“LLM+工具”的范式催生了工具增强型智能体的快速发展。

在数据分析领域，传统方法依赖于人工编写脚本或使用专业软件（如 Excel、SPSS、Tableau 等）进行数据清洗（Data Cleaning）、统计分析（Statistical Analysis）与可视化（Visualization）。这一过程往往需要较强的编程能力与领域知识，且难以应对非结构化需求。而数据分析智能体（Data Analysis Agent）则通过自然语言理解用户意图，自主生成并执行代码，将复杂的数据分析任务转化为可自动化完成的流程。

工具调用（Tool Use）是智能体的核心能力之一。LLM 本身无法直接执行代码或访问外部系统，但可以通过生成结构化的工具调用请求（如函数调用、API 请求等），将任务委托给专门的执行器（Executor）完成。这种“思考-行动-观察”的循环模式在 ReAct 框架（Reasoning and Acting）中得到了系统性阐述。ReAct 由 Yao 等人于 2022 年提出（论文《ReAct: Synergizing Reasoning and Acting in Language Models》），其核心思想是将推理轨迹（Reasoning Traces）与行动步骤（Action Steps）交替进行，使模型能够在执行过程中动态调整策略，处理多步骤、需要外部反馈的复杂任务。

具体而言，ReAct 框架包含三个关键步骤：Thought（思考）阶段，模型基于当前状态生成推理过程，分析下一步应该采取的行动；Action（行动）阶段，模型生成具体的工具调用指令，如执行 Python 代码、查询数据库或调用 API；Observation（观察）阶段，执行器返回操作结果，模型根据反馈更新状态并继续推理。这一循环可以持续多轮，直到任务完成。ReAct 框架已被广泛应用于问答系统（Question Answering）、代码生成（Code Generation）、网页导航（Web Navigation）等场景，是构建自主智能体的重要理论基础。

在数据分析场景中，智能体的典型工作流程为：首先接收用户的自然语言查询（如“分析销售数据的季节性趋势”），然后理解表格结构与字段含义，接着生成 Python 代码进行数据处理（如使用 pandas 进行分组聚合），在沙盒环境（Sandbox Environment）中安全执行代码，解析执行结果并生成自然语言回复或可视化图表，最后根据反馈迭代优化分析逻辑。代表性工作包括 OpenAI 的 Code Interpreter、Google 的 Codey 以及 Meta 的 LlamaIndex 等，它们均展示了 LLM 在数据分析任务中的强大潜力。

实验内容

本次实验将围绕数据分析智能体展开，核心目标是学习并实现 ReAct 工具调用流程，使大语言模型能够自主完成表格数据的分析任务。实验设计参考了 InfiAgent-DABench 的评估框架，该基准测试是首个专门用于评估基于大语言模型的数据分析智能体的测试集。实验将使用 Qwen2.5-7B 作为推理引擎，Python 沙盒 API 作为代码执行环境。

在本次实验中，我们将提供 CSV 格式的数据文件以及对应的数据分析任务需求。学生需要实现基于 ReAct 框架的完整工具调用流程：首先构建合适的 Prompt，使模型能够理解任务并进行推理规划（Thought 阶段）；然后模型生成可执行的 Python 代码作为行动方案（Action 阶段），代码通常使用 pandas、numpy 等数据分析库；接着调用 Python 沙盒 API 安全执行生成的代码，沙盒会返回执行结果包括标准输出、错误信息或计算结果（Observation 阶段）；最后将执行结果反馈给模型，模型根据观察结果继续推理或给出最终答案。这一“思考-行动-观察”的循环可能需要进行多轮迭代，直到任务完成。在实现过程中，学生需要设计

循环终止条件，如检测到模型输出"任务完成"标记、达到预设的最大迭代次数、或识别出模型陷入重复循环等情况。同时还需要实现健壮的异常处理机制：当代码执行出现语法错误或运行时异常时，将详细的错误信息作为 Observation 反馈给模型，引导其分析错误原因并生成修正代码；当执行超时或触及资源限制时，提示模型优化算法或简化计算策略；对于模型反复生成错误代码的情况，引入强制终止机制避免资源浪费。

实验任务将覆盖数据分析中的多个核心概念，包括描述性统计（Summary Statistics）如计算均值、中位数、标准差等基础指标；数据筛选与条件查询，如"查找满足特定条件的记录"；分组聚合操作（Groupby and Aggregation），如"按类别统计总销售额并排序"；相关性分析（Correlation Analysis），如"计算两个变量之间的皮尔逊相关系数"；异常值检测（Outlier Detection），如"使用 Z-score 方法识别异常数据点"；特征工程（Feature Engineering），如"创建新特征并分析其与目标变量的关系"；以及综合性的数据预处理与分析任务。这些任务的难度呈梯度分布，从简单的单步计算到复杂的多步骤综合分析，旨在全面评估智能体的数据处理能力、代码生成质量、错误恢复能力以及任务规划水平。

学生需要记录完整的实验过程，包括每一轮的 Thought 推理内容、生成的 Action 代码、Observation 执行结果，以及模型的最终回答。实验结束后，需要对智能体的性能进行量化评估，统计任务完成率（成功完成任务的比例）、代码正确率（首次生成的代码无需修改即可正确执行的比例）、平均迭代轮数（完成任务所需的平均 Thought-Action-Observation 循环次数）等关键指标。此外，还需要对失败案例进行深入分析，识别导致任务失败的典型模式，如模型对某些 pandas 函数理解不足、推理逻辑出现偏差、或对数据结构判断错误等。基于这些分析，学生可以尝试优化 Prompt 设计，如添加少样本示例（Few-shot Examples）帮助模型理解任务模式、引入思维链（Chain-of-Thought）提示促进更细致的推理过程、或在系统提示中详细说明可用的工具和函数以减少试错成本。通过本次实验，学生将深入理解智能体如何通过迭代式推理与外部工具反馈逐步解决复杂任务，掌握大语言模型在实际数据分析场景中的应用能力，并培养系统性的问题诊断与优化思维。

实验要求

1. 理解大语言模型与工具调用的基本概念，了解 ReAct 框架的核心思想与技术细节；
2. 学会使用 Qwen2.5-7B API 和 Python 沙盒 API，完成基础的代码生成与执行测试；
3. 掌握 ReAct 工具调用流程的完整实现方法，包括 Prompt 设计、多轮交互逻辑与异常处理机制；
4. 完成至少 5 个不同难度的表格数据分析任务，记录详细的推理轨迹与执行日志；
5. 对实验结果进行深入分析，总结智能体的优势与不足，提出针对性的改进思路与优化方案。

实验环境

使用 Python 语言，调用 Qwen2.5-7B API（文本生成）与 Python 沙盒 API（代码执行）。推荐使用 Jupyter Notebook 或本地 Python 环境进行开发与调试。

参考资料

Python 数据分析基础：

- Pandas 官方文档：<https://pandas.pydata.org/docs/>

- 菜鸟教程: <https://www.runoob.com/python3/python3-tutorial.html>
- **Qwen2.5 模型:**
- Qwen2.5-7B 模型文档: <https://hf-mirror.com/Qwen/Qwen2.5-7B-Instruct>
- Qwen API 使用指南: <https://help.aliyun.com/zh/model-studio/developer-reference/api-details>

ReAct 框架:

- 论文原文: *ReAct: Synergizing Reasoning and Acting in Language Models* (Yao et al., ICLR 2023)

工具调用相关论文:

- *Toolformer: Language Models Can Teach Themselves to Use Tools* (Schick et al., 2023)
- *A Survey on Large Language Model based Autonomous Agents* (Wang et al., 2023)